

Software Documentation

Project Name: Flappy Bird (C++ / SFML)

Developer Name: Muhammad Huzaifa

Tools Used: Visual Studio 2022, SFML 2.6.2, C++17/20, Git/GitHub

Abstract

This project is a high-fidelity clone of the classic Flappy Bird game, implemented in C++ using the Simple and Fast Multimedia Library (SFML). The application demonstrates core game development principles, including real-time input handling, physics simulation (gravity), collision detection, and modular system design. The focus of the project is on writing clean, maintainable, and object-oriented code suitable for academic and professional portfolios.

Introduction

Overview

Flappy Bird is a side-scrolling mobile game where the player controls a bird, attempting to fly between rows of green pipes without hitting them.

Purpose

The primary purpose of this project is to apply Software Engineering principles to a real-world game development scenario, showcasing proficiency in C++ and SFML.

Why C++ and SFML?

- C++: Offers high performance and low-level memory management, essential for smooth real-time rendering.
- SFML: Provides a robust, cross-platform interface for graphics, windowing, and input, making it ideal for 2D game development.

—

Tools & Technologies

- **C++**: The core programming language used for logic and memory management.
- **SFML** (Simple and Fast Multimedia Library): Used for graphics rendering and window management.
- **Visual Studio Community 2022**: The IDE used for development and debugging on Windows.
- **GitHub**: Used for version control and project documentation.

Project File Structure

- **main.cpp**: The entry point of the application. It initializes the `sf::RenderWindow` and starts the Game loop.
- **Game.h / Game.cpp**: The heart of the system. Manages the game loop, state transitions, background rendering, and orchestration of other modules.
- **Bird.h / Bird.cpp**: Contains the Bird class. Handles bird-specific textures, animations, and physics (gravity/flapping).
- **Pipes.h / Pipes.cpp**: Contains the Pipes class. Manages the generation, movement, and cleanup of the green pipe obstacles.
- **Globals.h**: A centralized header for global constants like window dimensions and scale factors.

System Architecture

The system follows a classic Game Loop Architectural Pattern:

1. **Input**: The Game class polls events from the `sf::RenderWindow`.
2. **Update**: Physics and logic are updated based on the elapsed time (`deltaTime`).
3. **Draw**: All entities (Bird, Pipes, Background) are rendered to the back-buffer and then displayed.

The architecture is highly modular, where Game acts as the controller, while Bird and Pipes act as the model/view components.

Game Logic Explanation

Bird Movement and Gravity (Bird.cpp)

The bird's vertical movement is governed by a velocity variable `velocity_y`. In every update, gravity is added to the velocity:

```
velocity_y += gravity * dt.asSeconds();
```

When the player presses "Space", `flapBird` is called, setting the velocity to a negative value (moving upwards).

Pipe Generation and Movement (Pipes.cpp)

Pipes are generated at the right edge of the screen and moved leftwards. The Pipes class uses static textures to save memory. Pipes are stored in a `std::vector` inside the Game class and removed once they go off-screen.

Collision Detection (Game.cpp)

The game uses Bounding Box Collision Detection. The `sf::FloatRect::intersects` method is used to check if the bird's sprite overlaps with any of the pipe sprites or the ground.

Score Calculation

The score is incremented when the bird successfully passes the horizontal boundary of a pipe pair. A state flag `startMonitoring` ensures the score is only counted once per pipe.

Software Engineering Principles

1. Anticipation of Change

Implemented via `Globals.h`. Constants like `WIN_WIDTH`, `WIN_HEIGHT`, and `SCALE_FACTOR` are centralized. This allows for rapid adjustments to game difficulty or resolution without refactoring the core logic across multiple files.

2. Modularity

The project is divided into distinct modules (Bird, Pipes, Game). Each module is encapsulated in its own header and source file. This allows for independent testing and easier maintenance. For example, the pipe movement speed can be changed in `Pipes.cpp` without affecting any code in `Bird.cpp`.

3. Separation of Concerns

- **main.cpp**: Bootstrap only.
- **Game class**: Controls the "Flow" and "State" of the game.
- **Bird class**: Internal physics of the player.
- **Pipes class**: Logic of the obstacles.

This separation ensures that a bug in the collision logic doesn't corrupt the movement physics.

How to Run the Project

1. **Prerequisites**: Install Visual Studio 2022 (<https://visualstudio.microsoft.com/>) and download the SFML 2.6.2 Visual C++ 15 (2017) 64-bit (<https://www.sfml-dev.org/download/sfml/2.6.2/>) libraries.

2. **Setup**:

- Clone the repository.
- Open Flappy bird-C++ SFML.sln.
- Ensure the SFML include and lib paths are correctly set in the Project Properties.
- Copy the bin DLLs to your project output directory.

3. **Run**: Press F5 to build and run.

Limitations

- No sound effects or background music (ready for future integration).
- Single-level difficulty (does not increase over time).
- Basic UI (Mainly textual).

Future Enhancements

- Integration of SFML-Audio for flapping and collision sounds.
- High-score persistence using file I/O.
- Dynamic difficulty (pipes move faster as the score increases).

—

Conclusion

The project successfully implements a professional-grade Flappy Bird clone. By strictly adhering to Software Engineering principles like Modularity and Separation of Concerns, the codebase is robust, readable, and highly extensible.